KVFold

文章结构与演示介绍

Expanding Cache Capacity for LLM Agents with Lossless Compression

这篇 demo 文章围绕 Agent 反复使用长上下文时的 KV 容量问题展开：通过无损压缩，让固定的 GPU 和外部缓存预算容纳更多 KV，再用前缀复用、并发请求和历史会话三个场景展示它带来的变化。

文章怎样组织

Section	Main content
I. Introduction	从编程 Agent 继续任务的例子引出问题；交代前缀缓存、容量瓶颈和相关工作，再概括 KVFold 的方案与主要结果。背景知识和研究现状整合在这一节。
II. System Overview	介绍两条压缩路径，以及 KV 的保存、恢复和 attention 计算。Pool 在 GPU 中保存压缩记录；Tier 压缩外部缓存，保留原生 GPU 路径。
III. Demonstration	文章重点。通过三个控制台场景，依次展示前缀复用与恢复、GPU 容量对并发的影响、外部容量对历史留存的影响。
IV. Conclusion	归纳固定预算下增加 KV 容量的效果，以及它对请求等待和历史复用的意义。

演示怎样串起来

从一个排查 timeout 的编程任务开始：先建立上下文，再追加回归测试任务；随后增加同时运行的会话，最后回到之前的历史任务。三个场景分别回答“哪些前缀能复用”“GPU 能容纳多少活跃上下文”和“旧会话的 KV 还能否留在缓存中”。

> 场景一已通过真实模型和 LMCache 跑通，并保存了运行记录。场景二、三展示论文中的历史容量实验；页面选择器切换已有实验配置。

Pool 使用 gather + FA3 完成 prefill，并在 fused decode kernel 中读取压缩 KV。Tier 保持原生 GPU KV 与 attention，只压缩 DRAM/NVMe 中的缓存记录。这两个模式对应后面两类容量对比。

材料：英文论文第 8 版；控制台验证记录与真实页面截图。

1 场景一：继续任务，观察前缀复用

观众依次使用 Build、Append、Edit early 和 Restore。页面把操作与前缀复用条放在一起，并显示输入、生成结果、cached/computed tokens 和 engine TTFT。

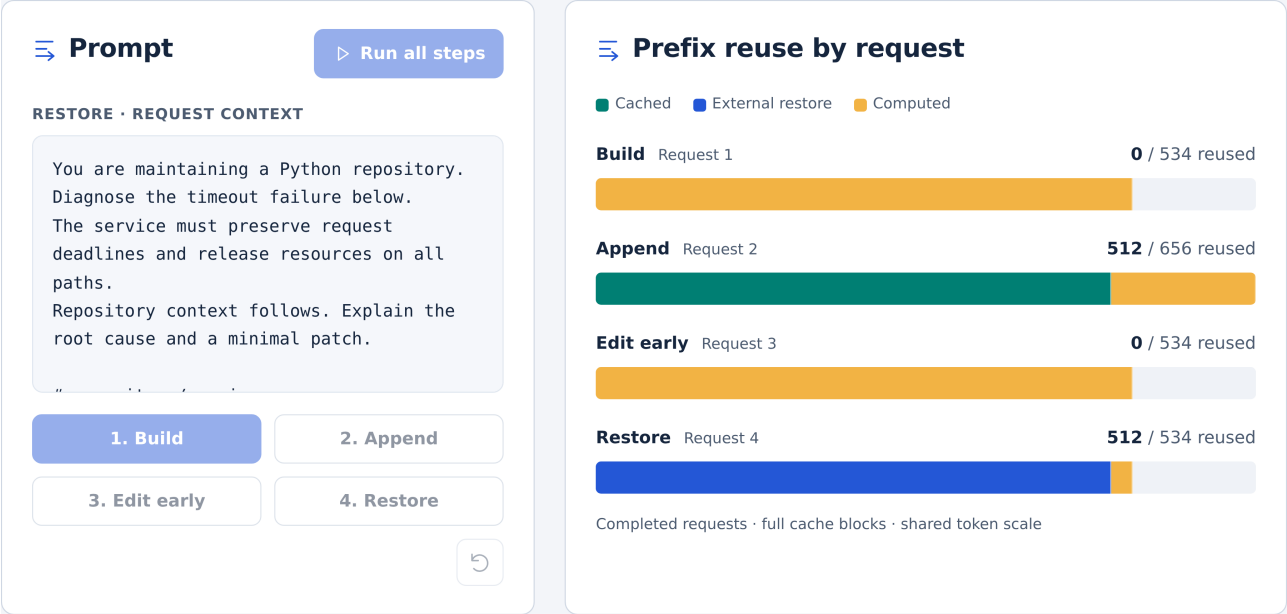


Figure 1. Prefix workflow from the recorded functional run.

Action	Interaction	Cached	Computed
Build	提交初始 timeout 排查任务，建立上下文	0	534
Append	保留原上下文和回答，追加回归测试任务	512	144
Edit early	修改开头内容，观察后续前缀复用失效	0	534
Restore	清空 GPU 前缀缓存，再请求原任务	512	22

Restore 已实际从 LMCache 的 CPU DRAM 取回 512 个 token。模型 KV 检查器另取一个已编码的完整 block，经过 GPU 导出、主机搬运和独立 GPU 区域导入后，全部层的 BF16 字节不一致数为 **0**。

> 这个场景让观众直接看到：追加任务可以复用前缀，靠前的修改会影响后续复用，GPU 中的前缀清除后还可以从外部缓存恢复。

功能验证：Qwen3-4B-Instruct-2507，单 H800 PCIe，TP=1，BF16 KV；4 GiB GPU KV、1 GiB LMCache DRAM。当前截图显示保存的真实运行；本轮外部恢复使用 DRAM，未启用 NVMe。

2 场景二：增加会话，观察 GPU 容量

观众切换并发准入上限 $S = 8$ 和 $S = 12$，比较同一 GPU KV 预算下的 Raw 与 Pool。页面同时显示可用 token 容量、等待请求数、engine TTFT 和 goodput。

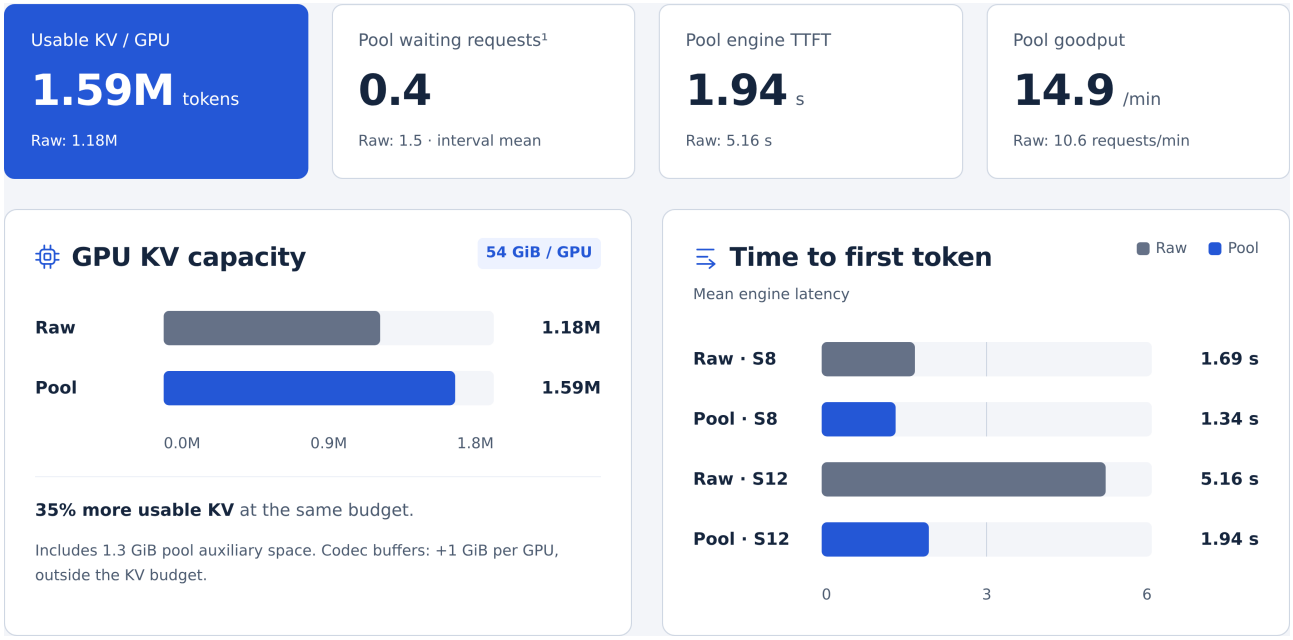


Figure 2. Recorded GPU-capacity comparison at admission cap S=12.

Metric	Raw	Pool
Usable KV capacity (M tokens)	1.18	1.59
Mean queue (requests), $S = 12$	1.5	0.4
Mean engine TTFT (s), $S = 12$	5.16	1.94
Goodput (requests/min), $S = 12$	10.6	14.9

演示先展示 $S = 8$，再切到 $S = 12$。Raw 的平均 TTFT 从 1.69 s 增至 5.16 s，Pool 从 1.34 s 增至 1.94 s。同样 54 GiB/GPU KV 预算下，Pool 可容纳的 token 增加约 34.7%；在 $S = 12$ 时，goodput 提高约 40.6%。

这一页的重点是容量增加如何缓解请求等待。四个配置的整体 cached-token share 均为 0.94，对比突出的是 GPU 中能同时容纳多少 KV。

历史 AgentX 实验：Qwen3-30B-A3B-Instruct，TP=2，双 H800；$P = 64$，400 GiB DRAM、2 TiB NVMe。Engine TTFT 不含代理排队时间；goodput 统计满足 TTFT $\leq$2 s、ITL $\leq$40 ms 的请求。

3 场景三：回到旧会话，观察历史留存

观众切换 640 GiB 和 2 TiB NVMe 预算，对比 Raw 与 Tier。两者保持相同的原生 GPU KV 和 attention 路径，观察压缩外部记录后能保留多少可复用历史。

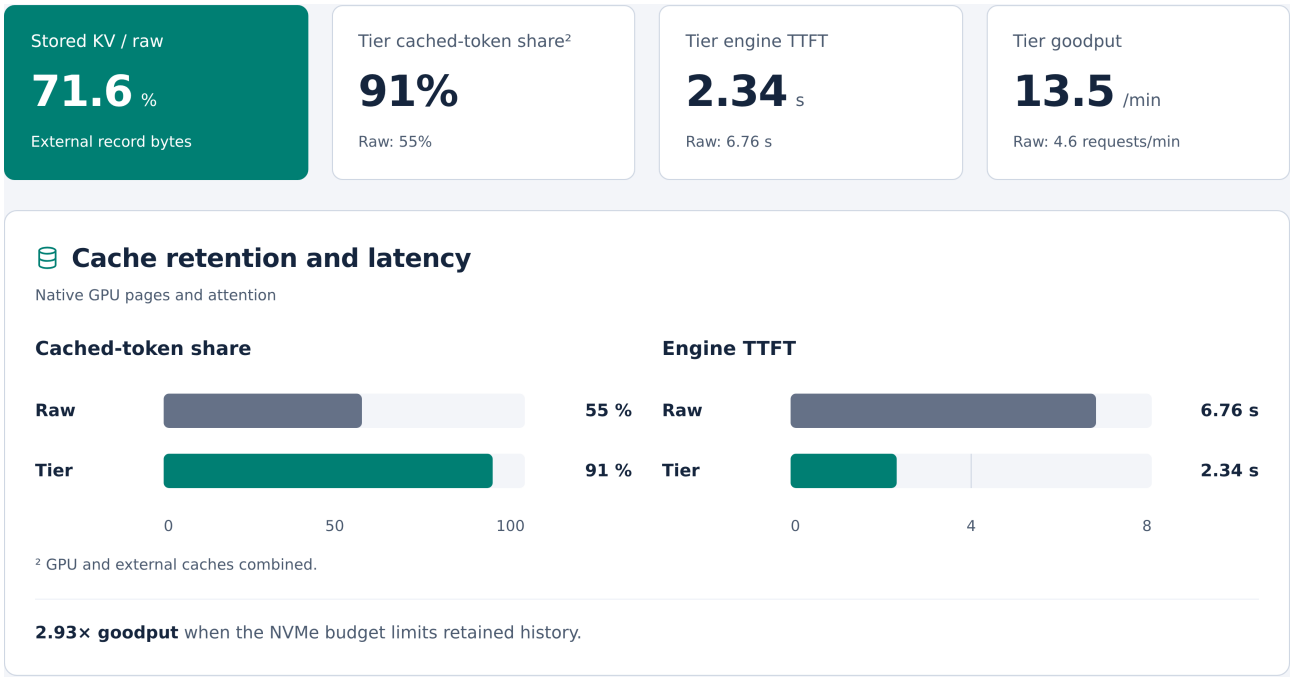


Figure 3. Recorded external-cache comparison at 640 GiB NVMe.

NVMe budget	Metric	Raw	Tier
640 GiB	Cached-token share	0.55	0.91
640 GiB	Mean engine TTFT (s)	6.76	2.34
640 GiB	Goodput (requests/min)	4.6	13.5
2 TiB	Cached-token share	0.94	0.94
2 TiB	Mean engine TTFT (s)	1.69	1.85

在 640 GiB 预算下，压缩让更多历史留在缓存中。切到 2 TiB 后，两者都能保留工作集，Tier 的 TTFT 略高，体现编解码开销。Cached-token share 统计 GPU 与外部缓存共同提供的前缀。

页面还用独立实验说明 DRAM 驻留：原始工作集为 261 GiB，压缩后为 188 GiB。在 220 GiB 预算下只有压缩数据能全部留在 DRAM；切到 400 GiB 后两者都能放下。

> 先展示容量紧张时保留更多历史的收益，再展示原始数据已经放得下时的成本，让观众理解压缩收益取决于具体的容量条件。

历史 AgentX 配置：$P = 64$、$S = 8$、400 GiB DRAM；DRAM 工作集示例来自另一组 48-prompt 实验。容量数据保留原报告的测量版本，早于最新码表与索引调整。